Start coding or generate with AI.

```
import sys
!{sys.executable} -m pip install transformers datasets torch pandas scikit-learn
print("Libraries installed successfully.")
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transfo
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from tran
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from trans
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-pac
Requirement already satisfied: typing-extensions>=4.10.0 in /usr/local/lib/python3.12/dist-packa
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from to
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch) (3
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pan
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from p
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from sci
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from sc
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-pack
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingfac
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from req
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from ai
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from ht
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpco
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typ
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-package
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markd
Libraries installed successfully.
```

```
from datasets import load_dataset

# Load the 'imdb' dataset
imdb_dataset = load_dataset('imdb')
```

```
# Print the dataset to inspect its structure
print(imdb_dataset)
print("Dataset loaded successfully.")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingfac
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datas
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable h
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
README.md: 7.81k/? [00:00<00:00, 466kB/s]
plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 105MB/s]
plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]
plain_text/unsupervised-00000-of-00001.p(...): 100% 42.0M/42.0M [00:00<00:00, 109MB/s]
Generating train split: 100% 25000/25000 [00:00<00:00, 102736.44 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 86400.38 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 120226.70 examples/s]
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})
Dataset loaded successfully.
```

```
from transformers import DistilBertTokenizerFast

# 1. Load the DistilBertTokenizerFast tokenizer
tokenizer = DistilBertTokenizerFast.from_pretrained('distilbert-base-uncased')

# 2. Define a tokenization function
def tokenize_function(examples):
    return tokenizer(examples['text'], truncation=True, padding=True)

# 3. Apply this tokenization function to all splits of the imdb_dataset
tokenized_datasets = imdb_dataset.map(tokenize_function, batched=True, remove_columns=['text'])

print("Dataset tokenization complete.")
print(tokenized_datasets)
```

```
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 2.12kB/s]

vocab.txt:  232k/? [00:00<00:00, 5.01MB/s]

tokenizer.json:  466k/? [00:00<00:00, 9.29MB/s]

Map: 100%  25000/25000 [00:32<00:00, 818.01 examples/s]

Map: 100%  25000/25000 [00:27<00:00, 879.99 examples/s]

Map: 100%  50000/50000 [01:02<00:00, 830.14 examples/s]
Dataset tokenization complete.
DatasetDict({
    train: Dataset({
        features: ['label', 'input_ids', 'attention_mask'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['label', 'input_ids', 'attention_mask'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['label', 'input_ids', 'attention_mask'],
        num_rows: 50000
    })
})
```

```
import torch

# 1. Extract the 'train' split
train_dataset = tokenized_datasets['train']

# 2. Extract the 'test' split (which is usually referred to as evaluation set in HuggingFace)
eval_dataset = tokenized_datasets['test']

# 3. Print the structure of both datasets
print("Train Dataset Structure:", train_dataset)
print("\nEvaluation Dataset Structure:", eval_dataset)
print("Datasets prepared successfully.")
```

```
Train Dataset Structure: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 25000
})

Evaluation Dataset Structure: Dataset({
    features: ['label', 'input_ids', 'attention_mask'],
    num_rows: 25000
})
Datasets prepared successfully.
```

```
from transformers import DataCollatorWithPadding
from torch.utils.data import DataLoader

# 1. Set the format of both train_dataset and eval_dataset to 'torch'
train_dataset.set_format(type='torch', columns=['input_ids', 'attention_mask', 'label'])
eval_dataset.set_format(type='torch', columns=['input_ids', 'attention_mask', 'label'])

# 2. Instantiate DataCollatorWithPadding
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)

# 3. Create DataLoader instances
batch_size = 8 # Adjust based on GPU memory
train_dataloader = DataLoader(train_dataset, shuffle=True, batch_size=batch_size, collate_fn=data
eval_dataloader = DataLoader(eval_dataset, batch_size=batch_size, collate_fn=data_collator)

print("PyTorch Datasets and DataLoaders created successfully.")
```

```
print(f"Train DataLoader batch size: {train_dataloader.batch_size}")
print(f"Evaluation DataLoader batch size: {eval_dataloader.batch_size}")
```

```
PyTorch Datasets and DataLoaders created successfully.
Train DataLoader batch size: 8
Evaluation DataLoader batch size: 8
```

```
from transformers import DistilBertForSequenceClassification

# 1. Load the pre-trained DistilBertForSequenceClassification model
# 2. Specify num_labels=2 for binary sentiment classification (positive/negative)
model = DistilBertForSequenceClassification.from_pretrained('distilbert-base-uncased', num_labels

print("Pre-trained DistilBertForSequenceClassification model loaded successfully.")
# 3. Optionally, print the model architecture to verify
print(model)
```

```
config.json: 100%                                                    483/483 [00:00<00:00, 38.0kB/s]

model.safetensors: 100%                                              268M/268M [00:01<00:00, 231MB/s]

Loading weights: 100%  100/100 [00:00<00:00, 412.07it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you exp
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider t
Pre-trained DistilBertForSequenceClassification model loaded successfully.
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

```
from transformers import DistilBertForSequenceClassification

# 1. Load the pre-trained DistilBertForSequenceClassification model
# 2. Specify num_labels=2 for binary sentiment classification (positive/negative)
model = DistilBertForSequenceClassification.from_pretrained('distilbert-base-uncased', num_labels

print("Pre-trained DistilBertForSequenceClassification model loaded successfully.")
# 3. Optionally, print the model architecture to verify
print(model)
```

```
Loading weights: 100%   100/100 [00:00<00:00, 426.49it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.w

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_transform.weight   | UNEXPECTED |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.weight        | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expe
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider tra
Pre-trained DistilBertForSequenceClassification model loaded successfully.
DistilBertForSequenceClassification(
  (distilbert): DistilBertModel(
    (embeddings): Embeddings(
      (word_embeddings): Embedding(30522, 768, padding_idx=0)
      (position_embeddings): Embedding(512, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
    )
    (transformer): Transformer(
      (layer): ModuleList(
        (0-5): 6 x TransformerBlock(
          (attention): DistilBertSelfAttention(
            (q_lin): Linear(in_features=768, out_features=768, bias=True)
            (k_lin): Linear(in_features=768, out_features=768, bias=True)
            (v_lin): Linear(in_features=768, out_features=768, bias=True)
            (out_lin): Linear(in_features=768, out_features=768, bias=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
          (ffn): FFN(
            (dropout): Dropout(p=0.1, inplace=False)
            (lin1): Linear(in_features=768, out_features=3072, bias=True)
            (lin2): Linear(in_features=3072, out_features=768, bias=True)
            (activation): GELUActivation()
          )
          (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        )
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

```
from transformers import TrainingArguments

# 2. Instantiate TrainingArguments
training_args = TrainingArguments(
    output_dir='./results',
    num_train_epochs=3,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    warmup_steps=500,
    weight_decay=0.01,
    logging_dir='./logs',
    logging_steps=10,
    # Temporarily removing evaluation_strategy and save_strategy due to TypeError,
    # and disabling load_best_model_at_end to resolve the subsequent ValueError.
    # Ideally, these would be set to 'epoch' and match.
    load_best_model_at_end=False, # Changed to False to bypass ValueError
```

```
    metric_for_best_model='accuracy',
    report_to='none'
)

print("Training Arguments defined successfully.")
# 3. Print the instantiated TrainingArguments object to verify the settings
print(training_args)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` in
Training Arguments defined successfully.
TrainingArguments(
accelerator_config={'split_batches': False, 'dispatch_batches': None, 'even_batches': True, 'use
adam_beta1=0.9,
adam_beta2=0.999,
adam_epsilon=1e-08,
auto_find_batch_size=False,
average_tokens_across_devices=True,
batch_eval_metrics=False,
bf16=False,
bf16_full_eval=False,
data_seed=None,
dataloader_drop_last=False,
dataloader_num_workers=0,
dataloader_persistent_workers=False,
dataloader_pin_memory=True,
dataloader_prefetch_factor=None,
ddp_backend=None,
ddp_broadcast_buffers=None,
ddp_bucket_cap_mb=None,
ddp_find_unused_parameters=None,
ddp_timeout=1800,
debug=[],
deepspeed=None,
disable_tqdm=False,
do_eval=False,
do_predict=False,
do_train=False,
enable_jit_checkpoint=False,
eval_accumulation_steps=None,
eval_delay=0,
eval_do_concat_batches=True,
eval_on_start=False,
eval_steps=None,
eval_strategy=IntervalStrategy.NO,
eval_use_gather_object=False,
fp16=False,
fp16_full_eval=False,
fsdp=[],
fsdp_config={'min_num_params': 0, 'xla': False, 'xla_fsdp_v2': False, 'xla_fsdp_grad_ckpt': Fals
full_determinism=False,
gradient_accumulation_steps=1,
gradient_checkpointing=False,
gradient_checkpointing_kwargs=None,
greater_is_better=True,
group_by_length=False,
hub_always_push=False,
hub_model_id=None,
hub_private_repo=None,
hub_revision=None,
hub_strategy=HubStrategy.EVERY_SAVE,
hub_token=<HUB_TOKEN>,
ignore_data_skip=False,
include_for_metrics=[],
include_num_input_tokens_seen=no,
label_names=None,
```

```
import sys
!{sys.executable} -m pip install evaluate
print(" 'evaluate' library installed successfully.")
```

```
Collecting evaluate
  Downloading evaluate-0.4.6-py3-none-any.whl.metadata (9.5 kB)
Requirement already satisfied: datasets>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from ev
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from evalua
Requirement already satisfied: dill in /usr/local/lib/python3.12/dist-packages (from evaluate) (0.
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from evaluate) (
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from e
Requirement already satisfied: tqdm>=4.62.1 in /usr/local/lib/python3.12/dist-packages (from evalu
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from evaluate) (
Requirement already satisfied: multiprocess in /usr/local/lib/python3.12/dist-packages (from evalu
Requirement already satisfied: fsspec>=2021.05.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: huggingface-hub>=0.7.0 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from evaluate
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from datasets>
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from da
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from datase
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packag
Requirement already satisfied: hf-xet<2.0.0,>=1.2.0 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from h
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from reque
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from panda
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from par
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from a
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aioh
Requirement already satisfied: frozenlist>=1.1.1 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (fro
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from a
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from http
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-da
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (fr
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (f
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdow
Downloading evaluate-0.4.6-py3-none-any.whl (84 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 5.1 MB/s eta 0:00:00
Installing collected packages: evaluate
Successfully installed evaluate-0.4.6
 'evaluate' library installed successfully.
```

```
import evaluate
import numpy as np

# Define a function to compute evaluation metrics
def compute_metrics(eval_pred):
    predictions, labels = eval_pred
    # Apply np.argmax to predictions to get the predicted class labels.
    predictions = np.argmax(predictions, axis=1)

    # Load the metrics
    accuracy_metric = evaluate.load('accuracy')
    precision_metric = evaluate.load('precision')
    recall_metric = evaluate.load('recall')
    f1_metric = evaluate.load('f1')

    # Compute each metric
    accuracy = accuracy_metric.compute(predictions=predictions, references=labels)['accuracy']
    precision = precision_metric.compute(predictions=predictions, references=labels, average='b
```

```
    recall = recall_metric.compute(predictions=predictions, references=labels, average='binary'
    f1 = f1_metric.compute(predictions=predictions, references=labels, average='binary')['f1']

    return {
        'accuracy': accuracy,
        'precision': precision,
        'recall': recall,
        'f1': f1
    }

print("Compute metrics function defined successfully.")
```

```
Compute metrics function defined successfully.
```